💻 Guía Práctica 7: Introducción a la Programación Orientada a Objetos

Facultad de Defensa Nacional – Licenciatura en Ciberdefensa
Materia: Lenguajes de Programación
Tema: Programación Orientada a Objetos (POO)
Lenguajes: Java y C++

🧩 Ejercicio 1

Enunciado:
Escribir una definición breve y concreta de “objeto” con un ejemplo del mundo real con su comportamiento y propiedades.

Respuesta:
Un **objeto** es una instancia de una clase que representa una entidad del mundo real, con **propiedades** (atributos) y **comportamientos** (métodos).

Ejemplo (Auto):

- **Propiedades:** color, marca, velocidad.
- **Comportamientos:** acelerar(), frenar(), encender().

🧩 Ejercicio 2

Enunciado:
Dado un sistema de biblioteca, modelar con UML al menos 3 posibles clases con su comportamiento y propiedades.

Solución (UML textual):

```
+-----------------+
|    Libro        |
+-----------------+
| - titulo        |
| - autor         |
| - isbn          |
+-----------------+
| + prestar()     |
| + devolver()    |
+-----------------+

+-----------------+
|    Usuario      |
+-----------------+
| - nombre        |
```

```
| - dni           |
| - nroSocio      |
+-----------------+
| + registrarse() |
| + reservarLibro()|
+-----------------+

+-----------------+
|   Biblioteca    |
+-----------------+
| - nombre        |
| - direccion     |
| - listaLibros   |
+-----------------+
| + agregarLibro()|
| + buscarLibro() |
+-----------------+
```

Ejercicio 3

Enunciado:
Escribir un pequeño programa estructurado que calcule el área de un rectángulo. Luego reescribirlo usando POO con una clase Rectángulo utilizando Java y C++.

Versión estructurada (C++)

```cpp
#include <iostream>
using namespace std;

int main() {
    float base, altura;
    cout << "Ingrese la base: ";
    cin >> base;
    cout << "Ingrese la altura: ";
    cin >> altura;

    float area = base * altura;
    cout << "El area es: " << area << endl;
    return 0;
}
```

Versión POO (C++)

```cpp
#include <iostream>
using namespace std;

class Rectangulo {
```

```
public:
    float base;
    float altura;

    float calcularArea() {
        return base * altura;
    }
};

int main() {
    Rectangulo r;
    r.base = 5;
    r.altura = 3;
    cout << "El area es: " << r.calcularArea() << endl;
    return 0;
}
```

◇ Versión POO (Java)

```
class Rectangulo {
    float base;
    float altura;

    float calcularArea() {
        return base * altura;
    }
}

public class Main {
    public static void main(String[] args) {
        Rectangulo r = new Rectangulo();
        r.base = 5;
        r.altura = 3;
        System.out.println("El area es: " + r.calcularArea());
    }
}
```

Ejercicio 4

Enunciado:
Implementar una clase Persona con atributos nombre, edad y dni. Crear un método presentarse() que imprima los datos de la persona.

C++

```
#include <iostream>
using namespace std;
```

```cpp
class Persona {
public:
    string nombre;
    int edad;
    string dni;

    void presentarse() {
        cout << "Hola, soy " << nombre
             << ", tengo " << edad
             << " años y mi DNI es " << dni << endl;
    }
};

int main() {
    Persona p;
    p.nombre = "Juan";
    p.edad = 25;
    p.dni = "12345678";
    p.presentarse();
    return 0;
}
```

💬 Java

```java
class Persona {
    String nombre;
    int edad;
    String dni;

    void presentarse() {
        System.out.println("Hola, soy " + nombre +
                           ", tengo " + edad +
                           " años y mi DNI es " + dni);
    }
}

public class Main {
    public static void main(String[] args) {
        Persona p = new Persona();
        p.nombre = "Juan";
        p.edad = 25;
        p.dni = "12345678";
        p.presentarse();
    }
}
```

🧩 Ejercicio 5

Enunciado:
Crear dos objetos Persona diferentes en main() y llamar al método presentarse() para ambos.

Java

```java
public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona();
        p1.nombre = "Juan";
        p1.edad = 25;
        p1.dni = "12345678";

        Persona p2 = new Persona();
        p2.nombre = "Maria";
        p2.edad = 30;
        p2.dni = "87654321";

        p1.presentarse();
        p2.presentarse();
    }
}
```

Ejercicio 6

Enunciado:
Agregar un método cumplirAnios() que aumente en 1 el valor de edad.

```java
class Persona {
    String nombre;
    int edad;
    String dni;

    void presentarse() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.");
    }

    void cumplirAnios() {
        edad++;
    }
}
```

Ejercicio 7

Enunciado:
Implementar un método esMayorDeEdad() que devuelva true o false según la edad.

```
boolean esMayorDeEdad() {
    return edad >= 18;
}
```

```
bool esMayorDeEdad() {
    return edad >= 18;
}
```

Ejercicio 8

Enunciado:
Implementar getters y setters en la clase Persona. Reescribir la clase Persona haciendo que sus atributos sean privados y accesibles solo mediante métodos get y set.

Java

```
class Persona {
    private String nombre;
    private int edad;
    private String dni;

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }

    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }

    public void presentarse() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.");
    }
}
```

Ejercicio 9

Enunciado:
Desde main intentar acceder a atributos privados directamente sin utilizar getters y setters.

```
Persona p = new Persona();
p.nombre = "Juan"; // ❌ Error: nombre tiene acceso privado
```

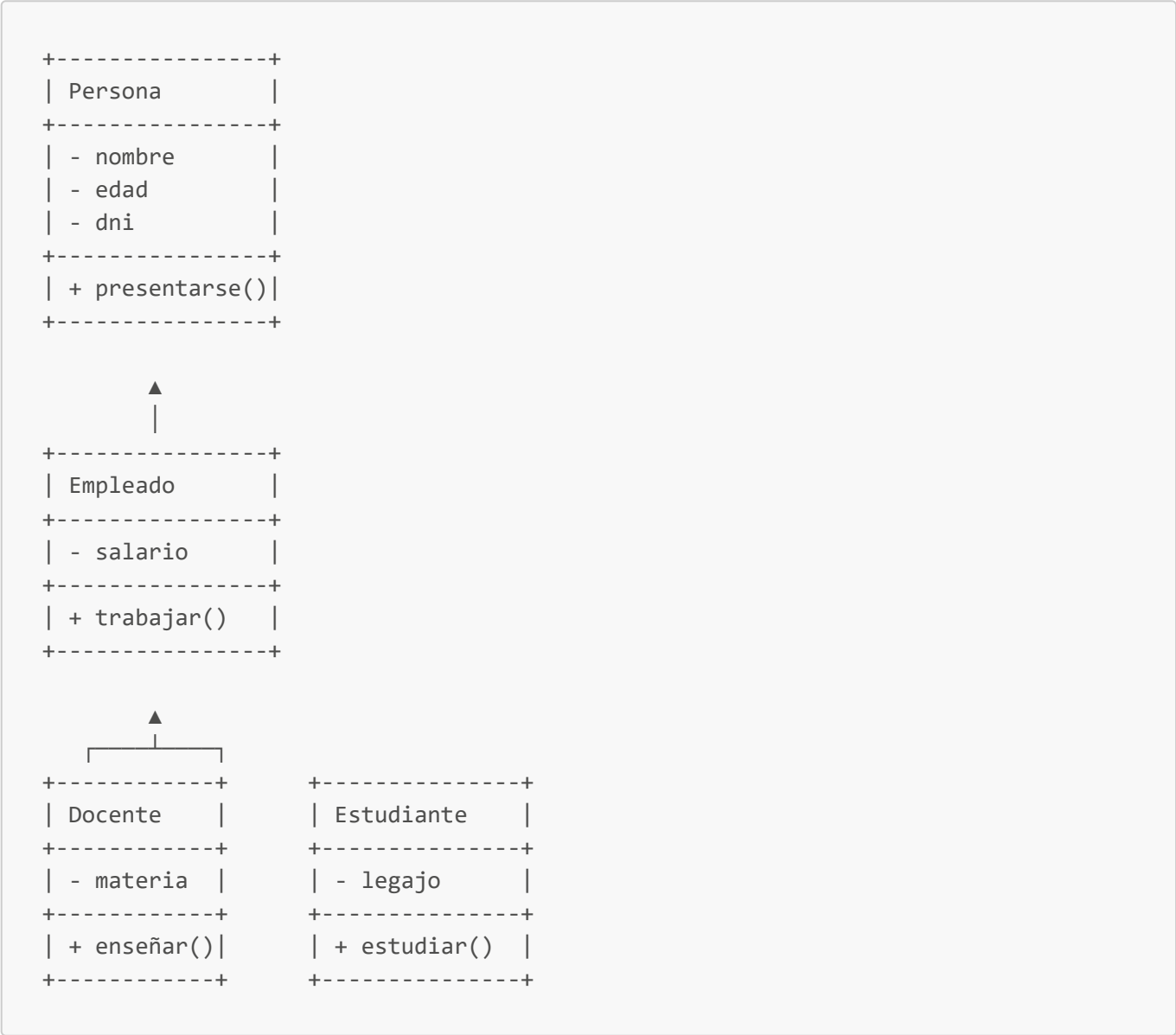
Forma correcta:

```
p.setNombre("Juan");
System.out.println(p.getNombre());
```

Ejercicio 10

Enunciado:
Modelar con UML una clase Persona, Empleado, Docente y Estudiante.

Diagrama UML:



Notas:

- Empleado hereda de Persona.
- Docente y Estudiante heredan de Empleado o directamente de Persona, según el diseño elegido.

Conclusión:

Esta guía permite comprender los fundamentos de la Programación Orientada a Objetos aplicando conceptos como **clases**, **objetos**, **encapsulamiento**, **métodos**, **atributos** y **herencia** en C++ y Java.